SDU Soft Robotics group meeting · October 2026

RESPONSIBLE AI FOR RESEARCH MEDIA

Plots, animations and synchronized video with Claude, MATLAB and Premiere Pro



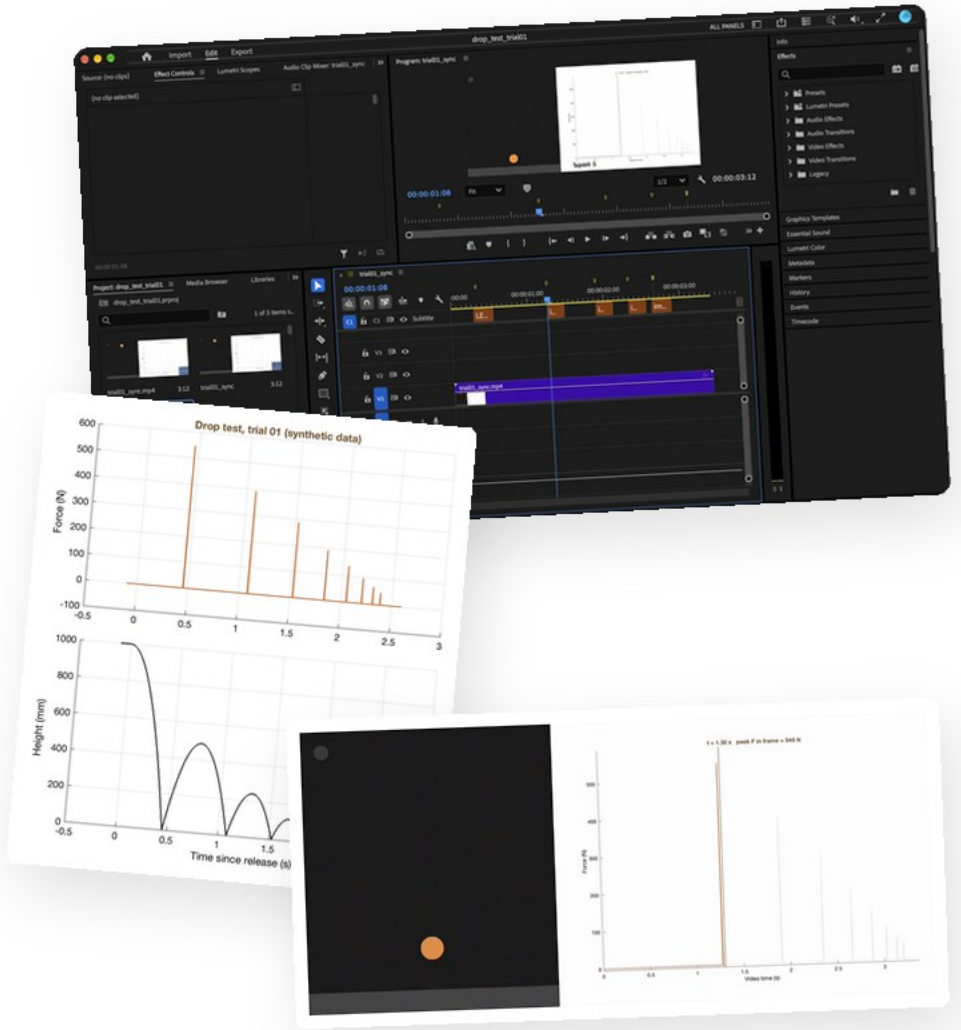
Jonathan Tirado

SDU Soft Robotics

University of Southern Denmark

AGENDA

1. **Responsible use**
   four rules and a human in the loop
2. **The toolkit**
   Claude app, desktop app, Claude Code and MCP connectors
3. **Plots and animations**
   MATLAB connector, publication figures, frame-by-frame video
4. **Video meets data**
   synchronizing camera and sensor clocks, a real case
5. **Premiere Pro**
   official and community routes, markers, guardrails
6. **Hands-on tutorial**
   the kit, prompts and checks to replicate it yourself



FOUR RULES BEFORE ANY PROMPT

Own the result



Claude drafts; **you sign**. Authorship, interpretation and errors stay with the researcher.

Verify every number



Re-run the script, compare with **raw data**, reject any value you can't trace.

Protect the data



Participant video and unpublished results need **consent** and an approved tool.

Disclose AI use



State in methods and captions where AI wrote **code**, edited **media** or drafted text.

Idea: AI speeds up the work; the evidence stays yours.

KEEP A HUMAN IN THE LOOP

1 · You	2 · Claude	3 · You	4 · You	5 · Both
Describe	**Draft**	**Run and read**	**Verify**	**Record**
Goal, data, units, constraints	Code, figure, edit plan	Understand every line	Against raw data and theory	Script, prompt, version saved

- Claude makes **step 2** fast
- Steps 1, 3 and 4 are where your **expertise and accountability** live

ONE ASSISTANT, THREE WAYS IN

Claude app



Web · desktop · mobile

Chat, analysis and finished outputs: slides, documents, dashboards. Projects keep a study's files together.

Claude Code



Terminal · IDE · desktop

An agent that reads your project folder, writes and runs scripts, fixes errors and keeps changes in git.

Connectors (MCP)



Bridges to other software

MATLAB, Adobe Creative Cloud, Blender and more: Claude calls their tools, with your permission.

Today's demos use all three.

THE DESKTOP APP

Your folders

Connect a data folder; Claude reads and writes files there, asking before it **deletes** anything.

Projects

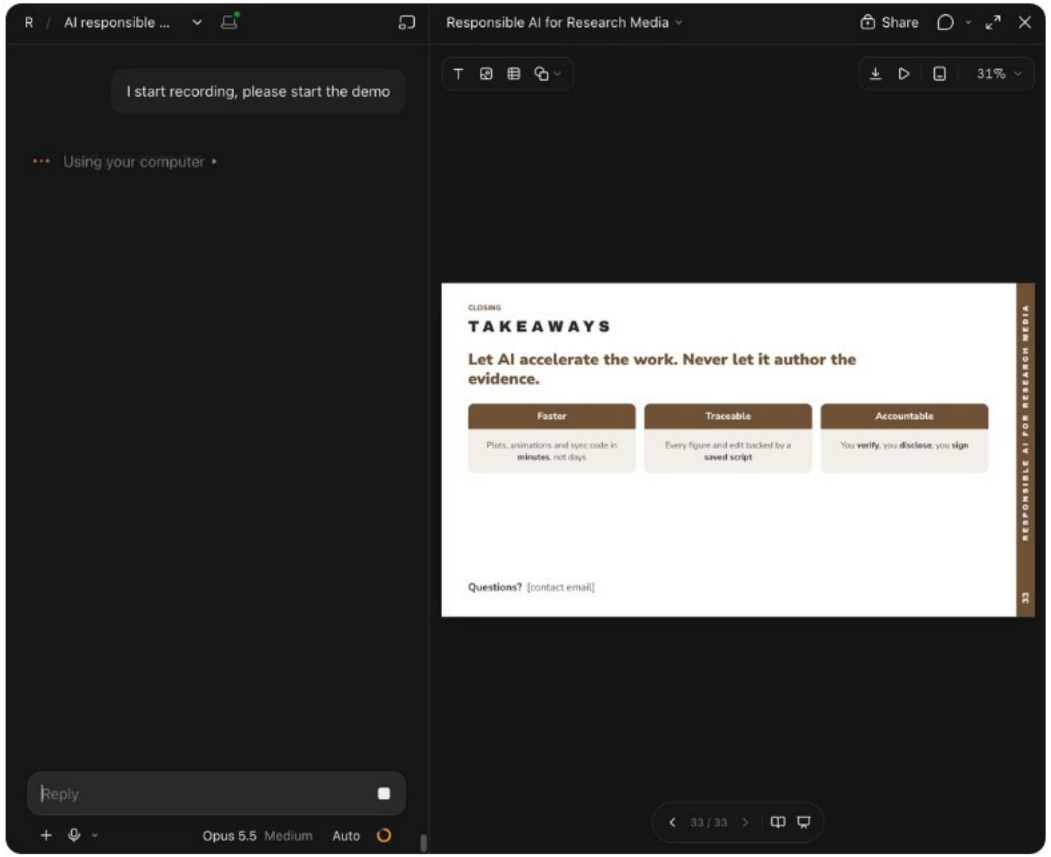
One space per experiment: **protocol, scripts and notes** reused across every chat.

Artifacts

Slides like these, documents, dashboards and small tools you can **edit and share**.

Connectors and apps

Install MATLAB or Adobe connectors, and let Claude operate desktop apps **when you allow it**.



The desktop app while preparing this talk: chat on the left, this deck as an artifact on the right

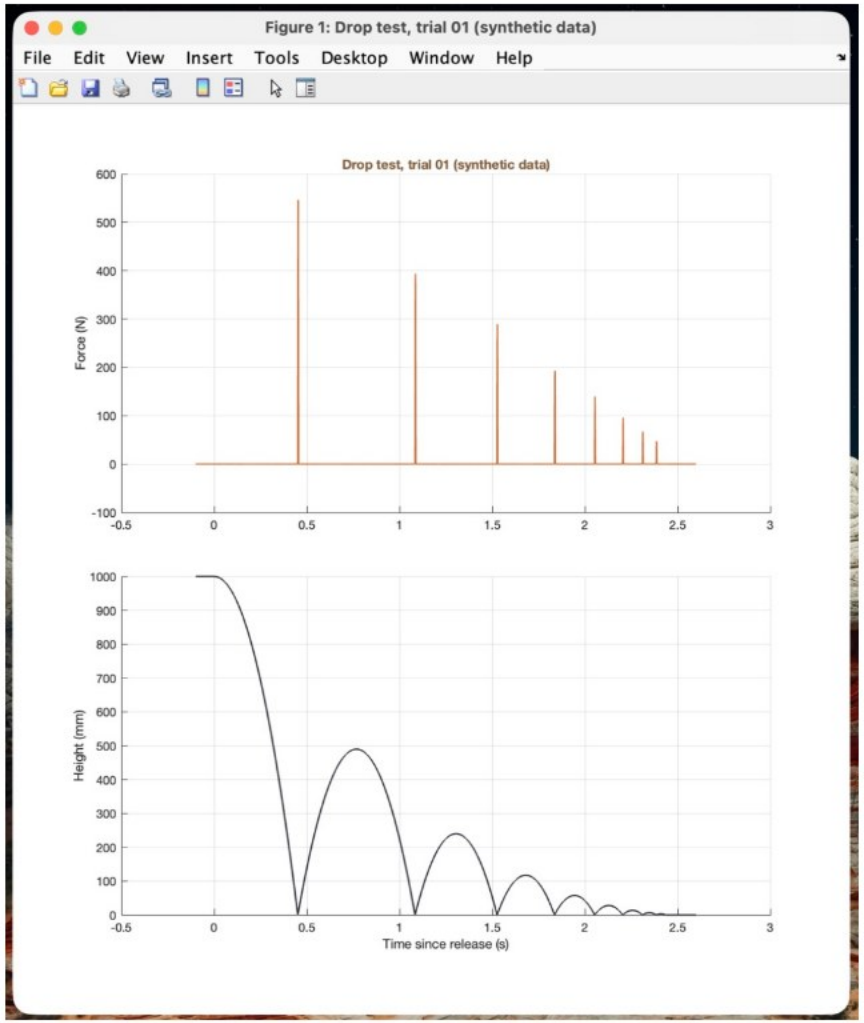
CLAUDE + MATLAB, FOR REAL

1 · Connector finds MATLAB

```
MATLAB Version 24.1 (R2024a) Update 3
Signal Processing · Curve Fitting · Image Proc. …
```

2 · Claude runs code in MATLAB (approved)

```
T = readtable("trial01.csv");
tiledlayout(f,2,1,"TileSpacing","compact")
nexttile; plot(tp, T.force(sel)); ylabel("Force (N)")
nexttile; plot(tp, 1e3*T.h(sel)); ylabel("Height (mm)")
exportgraphics(f,"fig_trial01.pdf","ContentType","vector")
> first rebound peak: 490 mm (e² × 1000 = 490 mm)
> Saved fig_trial01.pdf (vector, 85 mm)
```



Screenshot of the MATLAB window Claude opened

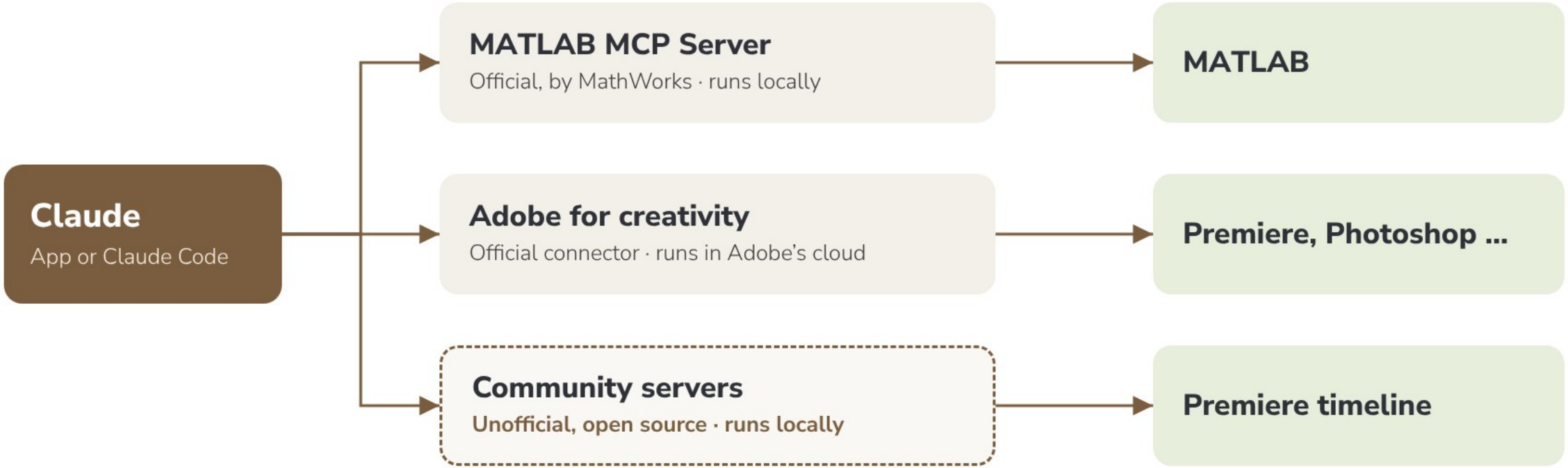
CLAUDE CODE

- An **agent in your project folder**: reads code and data before acting
- Writes, runs and **debugs** MATLAB or Python scripts
- **Asks permission** before editing files or running commands
- Works with **git**, so every change is reviewable

```
$ cd ~/experiments/drop-test
$ claude
› Load every CSV in data/raw, plot force vs
  time per trial, flag trials with dropped
  samples, save figures to figs/ as PDF.
  Reading 24 files in data/raw …
  Writing plot_trials.m …
  Allow running MATLAB? (y/n)
```

Illustrative session; output varies.

MCP: ONE PROTOCOL, MANY TOOLS



Every connection is a permission you grant, and can revoke.

CONNECTING CLAUDE TO MATLAB

1. Install MATLAB R2020b or later and add it to your PATH
2. Download the official MATLAB MCP Server from MathWorks on GitHub
3. Desktop app: double-click the **.mcpb** bundle, choose Install. Claude Code: one command
4. Ask: *"Run this in MATLAB and show me the figure."*

```
# Claude Code
claude mcp add --transport stdio \
  matlab -- /path/to/matlab-mcp-server \
  --initial-working-folder=~/experiments
```

⚠ Runs code on your machine and may send **anonymised usage data** to MathWorks by default. Check before using lab computers.

FROM PROMPT TO FIGURE

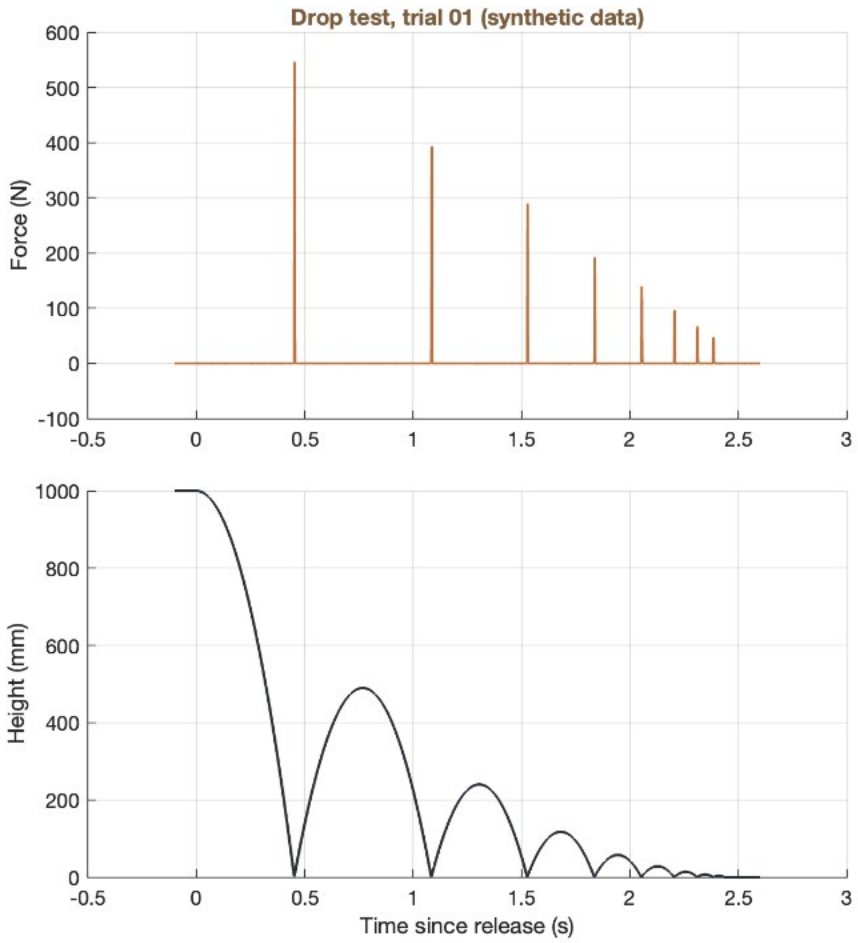
Your prompt

"Plot force and displacement from trial01.csv in two stacked panels with a shared time axis, SI units, exported as a vector PDF 85 mm wide."

Keep the script, not just the image: the .m file is the figure's provenance.

```
T = readtable("trial01.csv");
tiledlayout(2,1,"TileSpacing","compact")
nexttile
plot(T.t, T.force, "LineWidth",1.2)
ylabel("Force (N)")
nexttile
plot(T.t, 1e3*T.disp, "LineWidth",1.2)
xlabel("Time (s)"), ylabel("Displacement (mm)")
set(gcf,"Units","centimeters","Position",[2 2 8.5 9])
exportgraphics(gcf,"fig_trial01.pdf",...
    "ContentType","vector")
```

THE FIGURE CLAUDE PRODUCED



- Simulated **1 m drop test**: plate force and ball height
- **8 impacts**, peaks falling from 546 N
- Made by Claude in **your** MATLAB, through the connector; one dataset feeds every case slide

Check: first rebound ≈ 490 mm = e^2 × 1 m, as physics predicts.

CHECK THE FIGURE

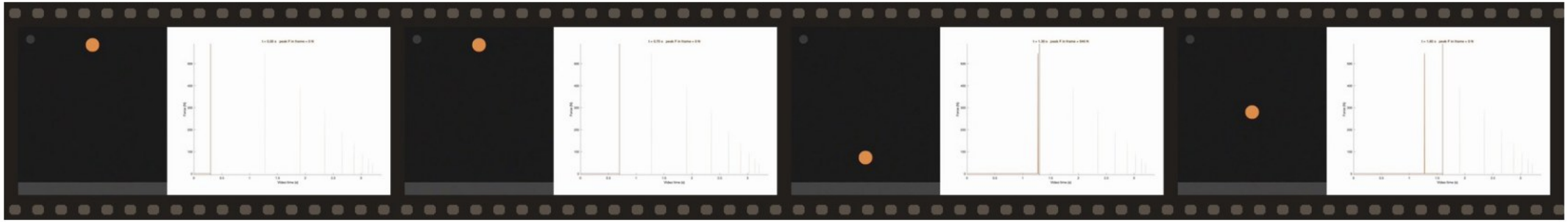
Check	What to look for
Provenance	Every value traces to the raw file through a saved script
Axes and units	Labelled, SI units, no truncated axis exaggerating an effect
Uncertainty	Error bars or bands say what they are: SD, SEM or CI
Processing	Filtering, smoothing and outlier removal stated and justified
Accessibility	Colour-blind-safe palette; series also differ by line style

Tip: ask Claude to list every transformation it applied, then check the list.

ANIMATING RESULTS

- Each frame is a plot; a **loop writes it to video**
- **Fix the axis limits**, so motion means data, not rescaling
- State the speed: **real time or slowed**, on screen and in the caption
- Same idea in Python (FuncAnimation) or ffmpeg; Claude Code writes any of them

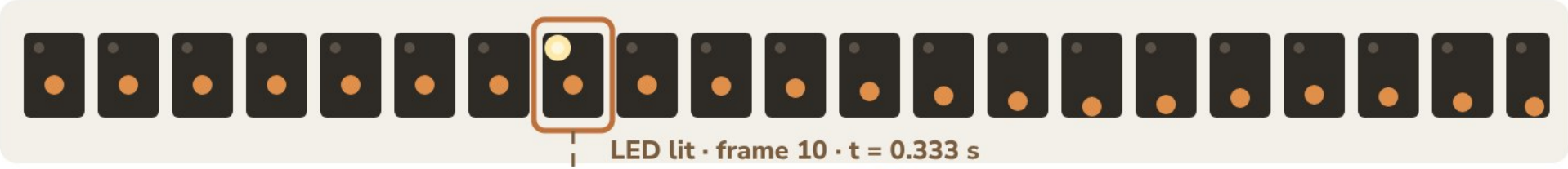
```
v = VideoWriter("wave.mp4","MPEG-4");
v.FrameRate = 30;
open(v)
fig = figure("Color","w");
for k = 1:numel(tFrames)
    plot(x, u(:,k), "LineWidth",1.5)
    ylim([-1 1])  % fixed axis
    title(sprintf("t = %.2f s", tFrames(k)))
    writeVideo(v, getframe(fig))
end
close(v)
```



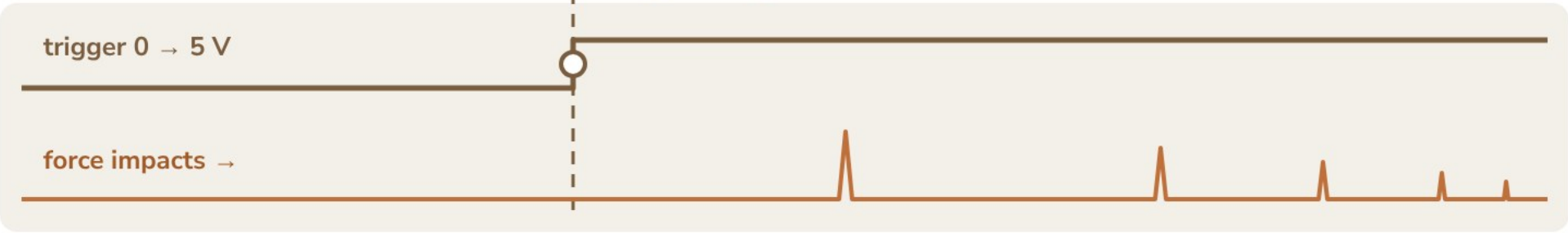
Real frames of trial01_sync.mp4 at t = 0.30, 0.70, 1.30 and 1.60 s, rendered by MATLAB

TWO CLOCKS, ONE SYNC EVENT

Camera
30 fps · video clock



Logger
1 kHz · own clock



1 · Detect	2 · Shift	3 · Sample	4 · Render
LED in the frame, step in the trigger	Logger time − 2.317 s = video time	Peak force within each frame	Plot beside each frame

ALIGNING THE CLOCKS

```
vr   = VideoReader("trial01.mp4");
tVid = (0:vr.NumFrames-1) / vr.FrameRate;
% shift sensor clock onto the video clock
tSig = T.t - tSyncSensor + tSyncVideo;
F    = interp1(tSig, T.force, tVid);
vw = VideoWriter("trial01_sync.mp4","MPEG-4");
vw.FrameRate = vr.FrameRate;  open(vw)
for k = 1:vr.NumFrames
    img = read(vr, k);
    pnl = renderPlot(tVid(1:k), F(1:k));
    writeVideo(vw, [img, pnl])
end
close(vw)
```

renderPlot is a helper you write (or ask Claude for) returning an image the same height as the frame.

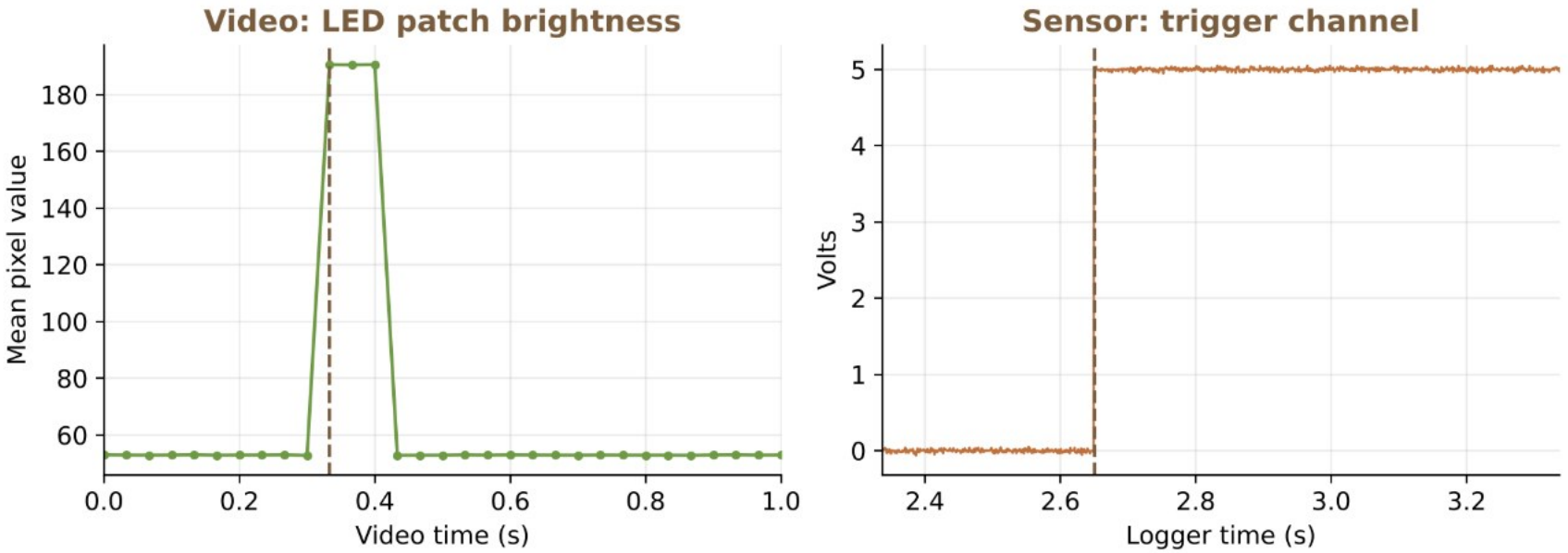
Watch out for

Variable frame rate phone video: use each frame's real timestamp

Clock drift in long runs: two sync events, correct the slope

Report the residual offset, e.g. ±1 frame = ±33 ms

FINDING THE SYNC EVENT

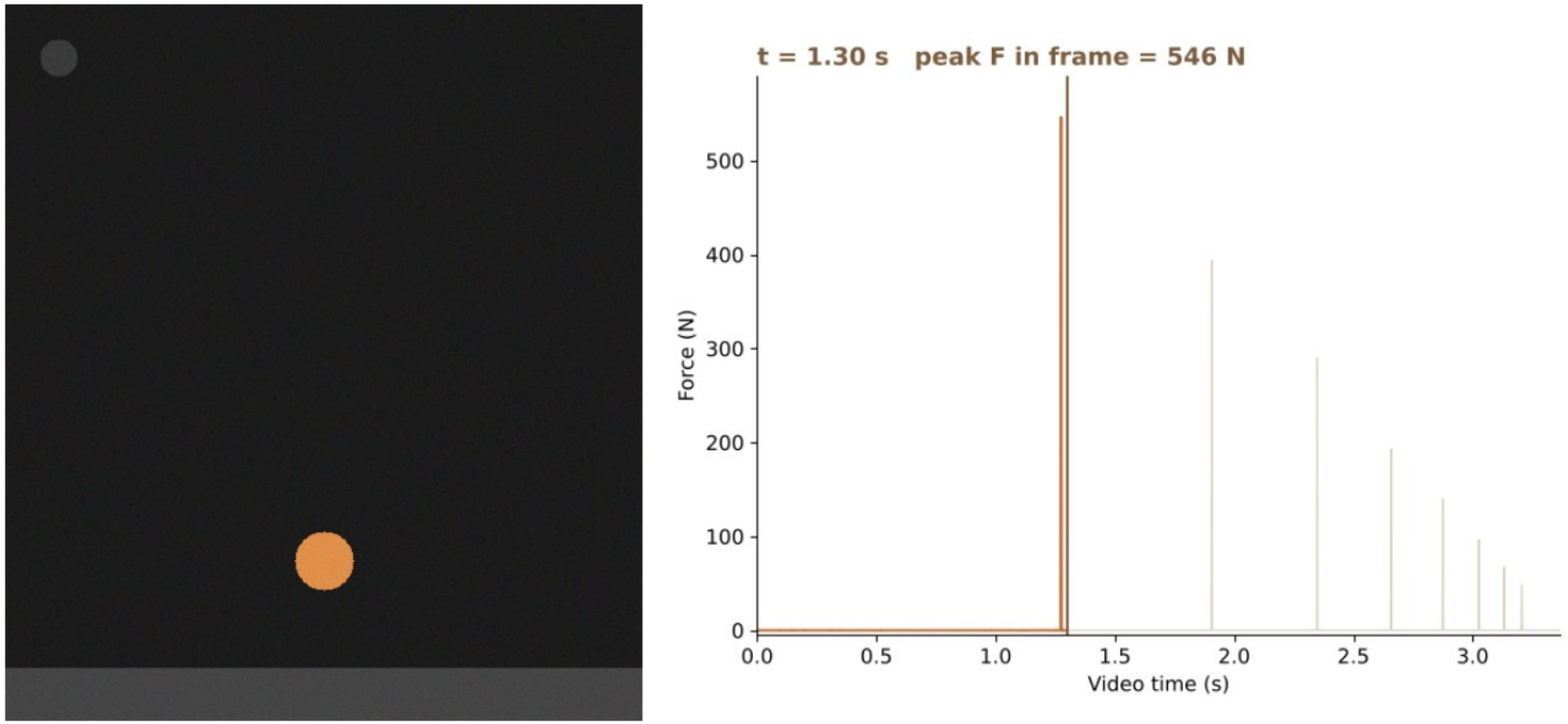


- LED flash found at **frame 10** (t = 0.333 s); trigger at logger t = 2.650 s
- Clock offset applied: 2.317 s

Residual vs truth: +20.3 ms, under one frame (33 ms).

THE SYNCED VIDEO

CAMERA · FORCE TRACE · t = 1.30 s, first impact



Camera frame on the left, force drawn up to this moment on the right · 102 frames rendered to trial01_sync.mp4

A QUIETLY WRONG NUMBER

First draft: value at each frame time

0 N

shown on the impact frame: wrong

Fix: peak within each frame interval

546 N

matches the raw 1 kHz record: right

- The impact lasts **4 ms**; a frame lasts **33 ms**: interpolation at frame times skipped it
- No error, plausible video: only **comparing against raw data** revealed it

THREE ROUTES INTO PREMIERE

Official connector	Community MCP servers	Scripts and files
Adobe for creativity (April 2026): 50+ Creative Cloud tools incl. Premiere, run in Adobe's cloud	Open-source bridges (**CEP panel** + **ExtendScript**) that drive your local timeline	Claude Code writes **ExtendScript** or caption/marker files; you run or import them
Official, no install, broad toolset	Full control of the local timeline	Every step reviewed by you
Footage goes to the cloud; coarse edits	Unofficial; may need unsigned extensions	Slower; manual import

EXPERIMENT TO ANNOTATED VIDEO

1	Render the synced plot and video, **trial01_sync.mp4**	MATLAB
2	Read the event log, **events.csv**: sync, impacts	Claude
3	Import raw and synced clips into a **new sequence**	Claude via MCP
4	Add a **marker and caption** at each logged event time	Claude via MCP
5	**Review** every cut and caption, then export the master	You

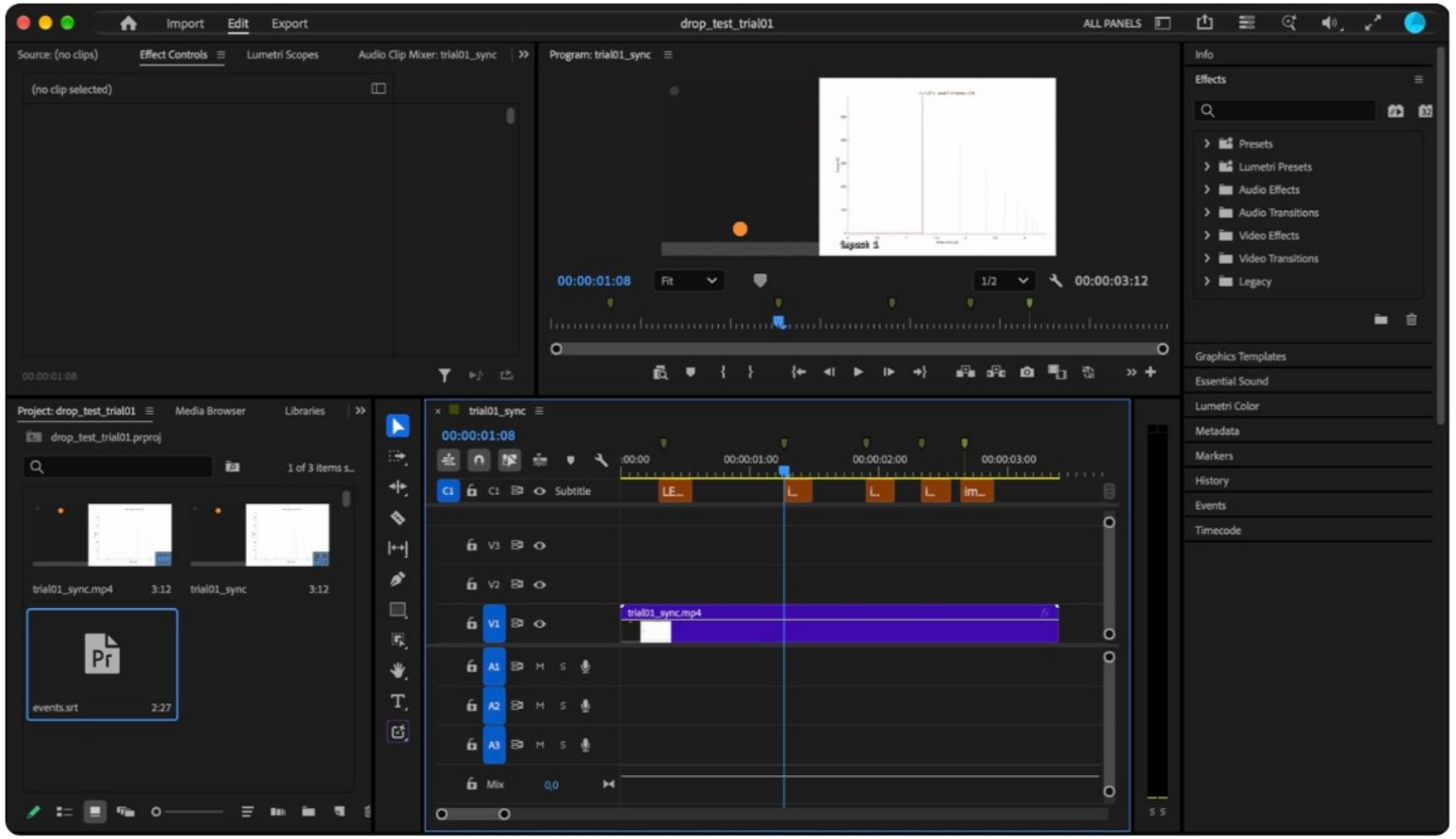
THE EVENT LOG

Event	Video time (s)
LED sync	0.333
Impact 1	1.271
Impact 2	1.903
Impact 3	2.346
Impact 4	2.655

- events.csv, written by sync_video.m in MATLAB, already on the video clock
- In step 4, Claude places one marker and caption per row

You check each marker against the footage.

CAPTIONS AND MARKERS, FOR REAL



- C1: 5 captions from events.srt
- Ruler: 5 markers at the logged times
- Monitor: impact 1, spike under the cursor

Gotcha: choose “Source timecode”, not “Timeline start”.

AUTOMATION NEEDS GUARDRAILS

Grant the least access

Approve each action at first; use **“always allow”** only for tools you trust.

Work on copies

Duplicate the project and keep **raw footage read-only** before automating.

Know what you install

Some bridges need unsigned extensions and run any script: treat them like **shell access**.

Keep a record

Save **prompts, scripts and edit logs** with the dataset so the edit can be audited.

YOUR TURN: THE TUTORIAL KIT

```
ResponsibleAI_media_tutorial/
├─ README.md          start here
├─ PROMPTS.md         copy-paste prompts
├─ 01_setup/          install the connector
├─ 02_data/           generate_drop_test.m
├─ 03_figure/         plot_trial.m
├─ 04_sync_video/     camera + sync scripts
├─ 05_premiere/       captions, markers, steps
├─ reference/         expected outputs
└─ run_all.m          everything, no AI
```

- Same **synthetic drop test** as this talk: no lab data needed
- Every step: **a prompt**, a **reference script**, and a **check**
- Tested end to end in MATLAB R2024a

Get it:
github.com/Jonasjcmh/AI_for_Media_Edition

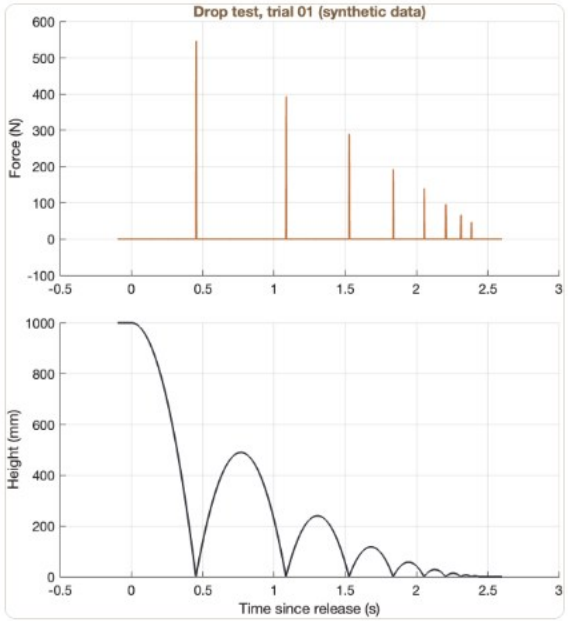
or the zip by email

INSTALL THE MATLAB CONNECTOR

1. Download **matlab-mcp-server.mcpb** from github.com/matlab/matlab-mcp-server (Releases)
2. Double-click it and click **Install** in the Claude desktop app
3. Settings → Extensions → MATLAB → Configure: **MATLAB Installation Path** = /Applications/MATLAB_R2024a.app
4. **Quit Claude fully (⌘Q)** and reopen it: closing the window is not enough

✓ Test: *"Which MATLAB version and toolboxes can you see?"* → your MATLAB version

PROMPTS: DATA AND FIGURE



What you should get: fig_trial01

Step 2 · Generate the dataset

"In MATLAB, simulate a drop test: 0.2 kg ball released from 1 m at t = 1 s, restitution 0.7, 4 ms half-sine impacts. Sample force at 1 kHz with 0.05 N noise (rng 7). Save trial01.csv and a logger.csv whose clock starts 2.137 s late, with a 5 V trigger at t = 0.513 s."

✓ **8 impacts · peak ≈ 546 N**

Step 3 · Publication figure

"Plot force and height from trial01.csv in two stacked panels, shared time axis, SI units, no smoothing. Export a vector PDF 85 mm wide. Tell me the first rebound height and ***list every transformation*** *you applied."*

✓ **First rebound ≈ 490 mm = e² × 1 m**

Full prompts in PROMPTS.md · reference scripts: generate_drop_test.m, plot_trial.m

PROMPTS: VIDEO AND SYNC

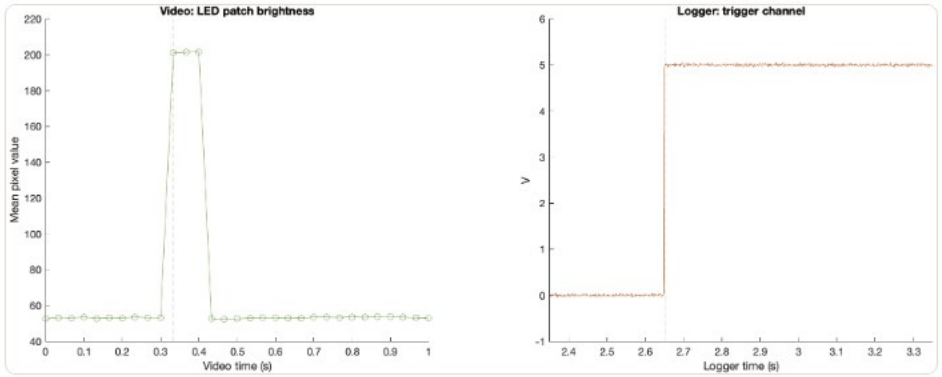
4a · Camera (synthetic)

"Render a 30 fps synthetic camera video camera.mp4 of the ball from trial01.csv, starting at t = 0.2 s, with an LED lit for 100 ms from t = 0.513 s."

Skip with real footage.

4b · Synchronise and render

*"Detect the LED flash in camera.mp4 and the trigger in logger.csv, shift the logger onto the video clock, take the **peak force within each frame**, and render trial01_sync.mp4 side by side. Save sync_check.png and events.csv, and compare interp1 vs peak on the impact frame."*



✓ Open **sync_check.png** (left) and confirm both detections by eye · offset ≈ 2.317 s · impact frame: interp1 ≈ 0 N, frame peak ≈ 546 N

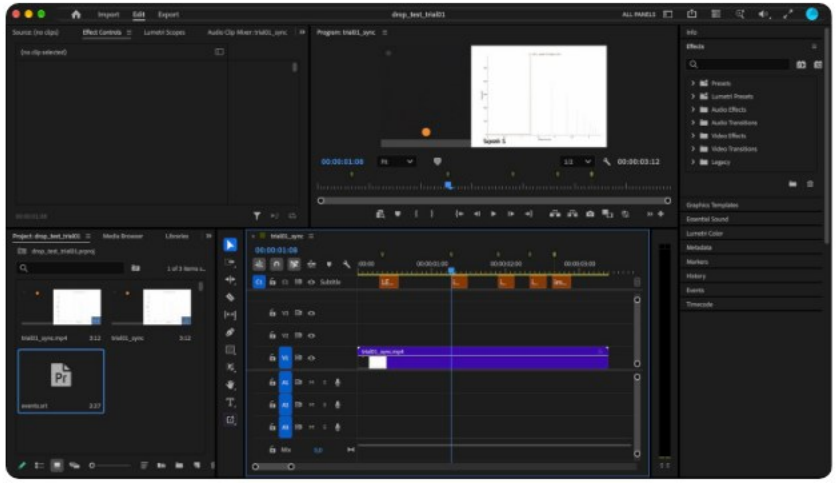
ANNOTATE IN PREMIERE PRO

Prompt

"Convert events.csv into an SRT caption file (0.25 s per event, no overlaps) and a list of HH:MM:SS:FF marker timecodes at 30 fps."

```
00:00:00:10  LED sync
00:00:01:08  impact 1
00:00:01:27  impact 2
00:00:02:10  impact 3
00:00:02:20  impact 4
```

1. Import **trial01_sync.mp4** · File → New → Sequence From Clip
2. Import **events.srt**, drag it to the Timeline, pick **Source timecode**
3. Click the playhead timecode, type it, then **Markers** → **Add Marker**
4. **Check**: on each impact marker the spike is under the cursor, ball at the floor
5. File → Export → Media (H.264)



Optional: a community Premiere MCP server can place the markers for you. Unofficial: see PREMIERE_STEPS.md.

VIDEO: YOU RECORD, CLAUDE ACTS

1

Open, then record

Open **Premiere**, close the start screen

⌘⇧5 → Record Selected Portion around it

Windows: Win+Alt+R

2 · Tell Claude

"I am recording my screen. Take control of Adobe Premiere Pro and repeat Step 5 in a new project called drop_test_demo: import trial01_sync.mp4, Sequence From Clip, drag events.srt with ***Source timecode****, add the markers from markers_list.txt, park on impact 1.* ***Pause about 2 s after each action.*** *Only use Premiere; ask me before anything else."*

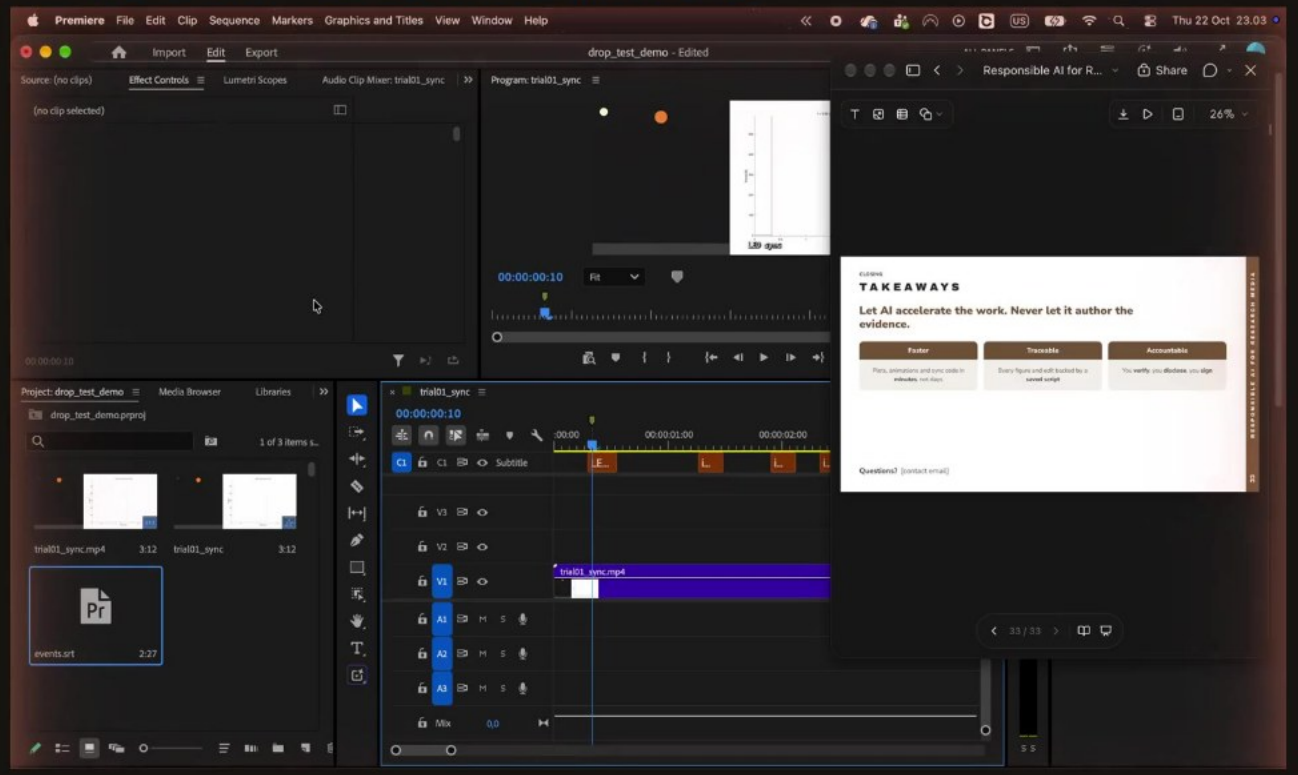
3 · Stop, then tell Claude

"Here is my recording. Trim the dead time, speed up waits ×4, caption each action, label it 'Claude controlling Premiere Pro · real recording · sped up', and export an MP4, 1080p, under 50 MB."

✓ Before: computer use on, other floating apps closed, Do Not Disturb on, hands off. After: watch it once; captions must match the actions.

WATCH CLAUDE WORK IN PREMIERE

Claude controlling Premiere Pro · real screen recording, 22 Oct 2026 ×4



13/14 Markers → Add Marker at each event time (5 markers)

- Two approvals before Claude touches anything
- 14 steps, 4 min real time, shown ×4
- Private details hidden: chat list cropped, folder names blurred

Plays when presented · click to pause

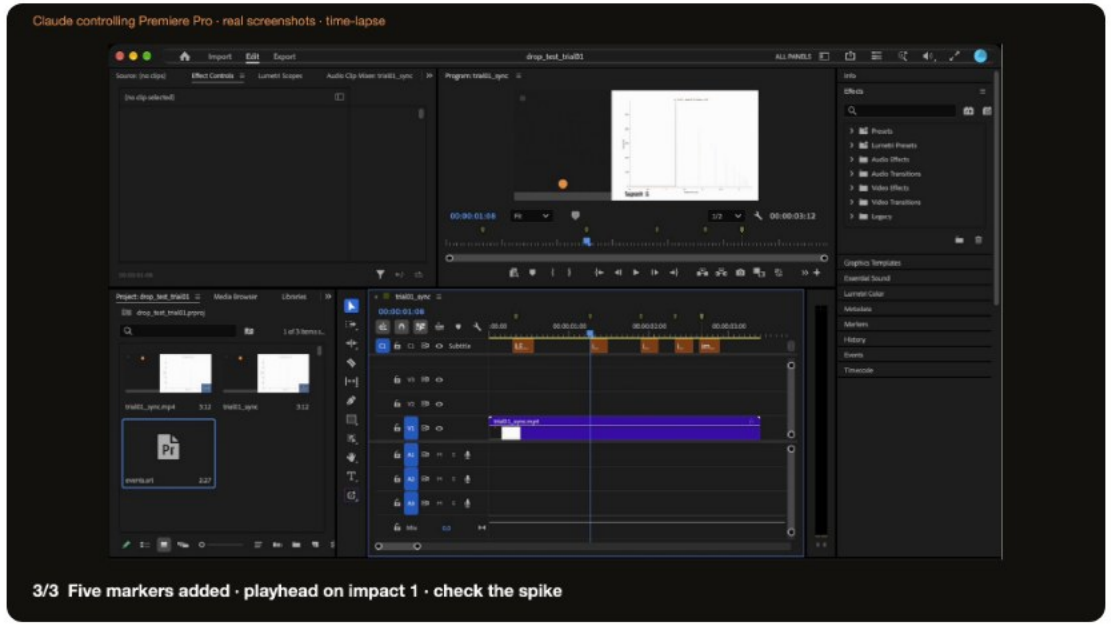
VIDEO: A TIME-LAPSE BY CLAUDE

1 · Tell Claude

*"Take control of Adobe Premiere Pro and repeat Step 5 in a new project called drop_test_demo. **After every action, save a numbered screenshot** in 06_screen_video/frames and write steps.csv with one short caption per screenshot. Include any mistake you make and how you fixed it."*

2 · Then

"Run make_timelapse.m in MATLAB and show me one frame of claude_controls_premiere.mp4."



A real frame from make_timelapse.m (sample run, 22 Oct 2026)

✓ Each caption matches its screenshot · the label says **AI-controlled** and **time-lapse** · nothing private on screen

YOU SHOULD SEE

Step	Output	Check
1	Connector answers	MATLAB version and toolboxes listed
2	trial01.csv, logger.csv	8 impacts, peak ≈ 546 N
3	fig_trial01.pdf	First rebound ≈ 490 mm; no smoothing applied
4	trial01_sync.mp4	Offset ≈ 2.317 s; impact frame peak ≈ 546 N, not 0 N
5	events.srt in Premiere	Captions on C1 match markers; spike under cursor
6	Screen video (optional)	Labelled AI-controlled; captions match actions

Different numbers? Compare with reference/, then ask Claude to explain the gap.

TAKEAWAYS

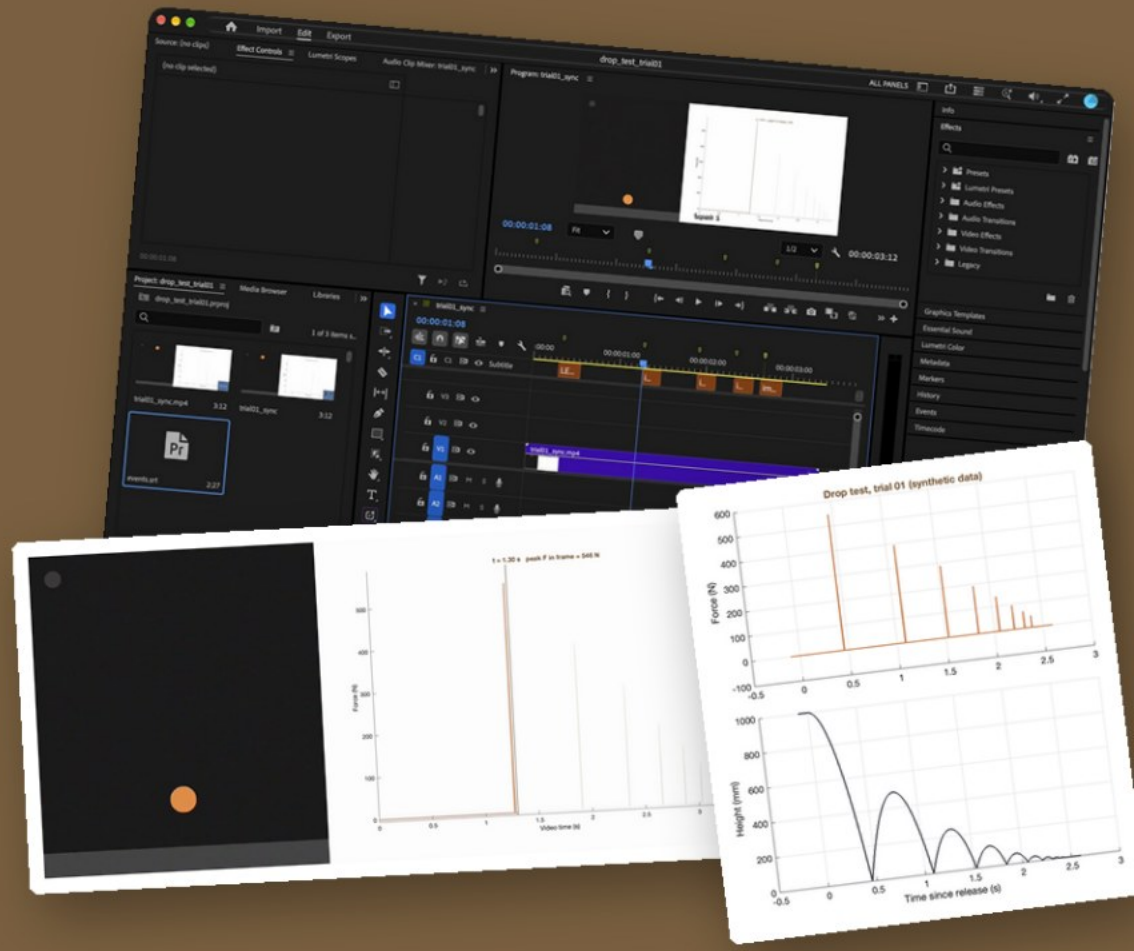
Let AI accelerate the work. Never let it author the evidence.

Faster	Traceable	Accountable
Plots, animations and sync code in **minutes**, not days	Every figure and edit backed by a **saved script**	You **verify**, you **disclose**, you **sign**

SDU Soft Robotics

THANK YOU ☺

Questions and feedback are very welcome



Jonathan Tirado
SDU Soft Robotics
joti@sdu.dk
Tutorial kit: github.com/Jonasjcmh/AI_for_Media_Edition

LinkedIn